Docker

GUIDE PRATIQUE

SecureShip · 22 situations, commandes et résultats attendus

Objectif : Apprendre Docker dans l'ordre où je l'utiliserai sur SecureShip : préparer les fichiers, construire et lancer l'application, gérer les conteneurs et préserver les données.
Environnement : MacBook, Docker Desktop, FastAPI, React, SQLite et Docker Compose.
Point de départ : Le projet existe déjà sur mon MacBook. J'ai créé à sa racine un `Dockerfile` et un `docker-compose.yml`, mais je n'ai encore construit aucune image ni lancé de conteneur SecureShip. Mon fichier local `.env` doit aussi être configuré.
Principe : Si les deux fichiers sont présents et que Compose est configuré pour construire l'image, je commence *directement* avec `docker compose up -d --build`. Je n'ai pas besoin d'exécuter `docker build` avant. Si je travaille seulement avec un `Dockerfile`, je peux utiliser `docker build` puis `docker run` : c'est une méthode alternative, présentée à la fin du chapitre.

Phase 1 — Préparer Docker et la configuration de SecureShip

1. Vérifier que Docker est installé sur mon MacBook

Si je veux savoir si Docker est installé sur ma machine et connaître sa version :

```
docker --version
```

Résultat attendu : Le terminal affiche `Docker version` et un numéro de version.

2. Vérifier que le moteur Docker fonctionne

Si Docker est installé, mais que je veux m'assurer que Docker Desktop est démarré :

```
docker info
```

Résultat attendu : Docker affiche des informations sur le client, le serveur, les images et les conteneurs. Si le moteur n'est pas démarré, la commande peut échouer avec une erreur de connexion au daemon.

3. Tester Docker avec un conteneur simple

Si je veux confirmer que Docker peut télécharger et exécuter une image, sans utiliser encore SecureShip :

```
docker run hello-world
```

```
Hello from Docker!
This message shows that your installation appears
to be working correctly.
```

Docker télécharge l'image si nécessaire, crée un conteneur, lance son programme, puis celui-ci se termine. Ce test est indépendant de notre application.

4. Vérifier les fichiers nécessaires à Docker Compose

Si je veux préparer le premier démarrage de SecureShip, je me place à la racine du projet et je vérifie que mes deux fichiers existent :

```
cd ~/Desktop/SecureShip
ls -l Dockerfile docker-compose.yml
```

Résultat attendu : Le terminal affiche les deux fichiers. Ils ont été écrits par le développeur ; ces commandes ne les créent pas.
Rôle de chacun :

— `Dockerfile` : explique comment construire l'image (compiler React, installer Python/FastAPI, copier l'application).
— `docker-compose.yml` : décrit le service `secureship`, le build, les ports, les volumes et les variables d'environnement.
— `.env` : contient la configuration locale d'authentification requise. Je ne dois pas le publier.

Si je veux vérifier la configuration Compose *sans afficher les valeurs sensibles* :

```
docker compose config --quiet
```

Résultat attendu : Aucun message lorsque la configuration est valide, sinon un message d'erreur. Cette commande ne construit et ne démarre rien.
Si je veux connaître les services déclarés dans le fichier Compose :

```
docker compose config --services
```

```
secureship
```

Attention : `docker compose config` sans `--quiet` peut afficher des valeurs sensibles provenant de l'environnement.

Phase 2 — Construire et démarrer avec Docker Compose

5. Construire et lancer SecureShip pour la première fois

Si j'ai les deux fichiers prêts, que Docker fonctionne et que rien n'a encore été créé pour SecureShip dans Docker, **je commence directement par** :

```
cd ~/Desktop/SecureShip
docker compose up -d --build
```

Résultat attendu : Docker Compose lit `docker-compose.yml`, utilise le `Dockerfile` pour construire l'image, crée le conteneur, le réseau et le volume nécessaires, puis démarre SecureShip en arrière-plan.

Comprendre cette commande :

— `up` : crée, démarre ou recrée les conteneurs nécessaires.
— `-d` : laisse l'application tourner en arrière-plan (*detached mode*).
— `--build` : demande de construire les images avant de démarrer.

Pourquoi je n'utilise pas d'abord `docker build` ? Parce que Compose s'occupe déjà du build grâce à la configuration de notre service. La construction de l'image n'a pas disparu : elle est effectuée par Compose.

Si je veux uniquement construire l'image, sans démarrer :

```
docker compose build
```

Si je veux lancer le service sans demander explicitement une reconstruction :

```
docker compose up -d
```

Compose peut néanmoins construire ou récupérer une image si elle manque. Pour une première exécution explicite et reproductible de SecureShip, je privilégie `up -d --build`.

L'application sera accessible, une fois démarrée et prête, à l'adresse :

```
http://127.0.0.1:8000/app
```

6. Retrouver l'image construite par Docker Compose

Si je veux voir les images actuellement disponibles sur mon MacBook après le build :

```
docker image ls
```

Résultat attendu : Un tableau affichant `REPOSITORY`, `TAG`, `IMAGE ID` et `SIZE`.

Important : Le nom et le tag de l'image créée par Compose dépendent de `docker-compose.yml`. Ils ne sont pas obligatoirement `secureship:dev` : ce tag appartient à notre exemple de build *manuel* présenté plus loin.

7. Vérifier que mon conteneur est en cours d'exécution

Si je veux voir l'état du service déclaré dans notre fichier Compose :

```
docker compose ps
```

```
NAME                    STATUS         PORTS
secureship-secureship-1  Up (healthy)   127.0.0.1:8000->8000/tcp
```

Exemple représentatif : Le nom exact du conteneur peut varier. `Up` signifie qu'il fonctionne ; `healthy` signifie que son contrôle de santé a réussi.

Si je veux voir les conteneurs actifs de *tous* mes projets :

```
docker ps
```

Si je veux également voir les conteneurs arrêtés :

```
docker ps -a
```

8. Vérifier que l'API répond réellement

Si SecureShip est démarré et que je veux tester son endpoint de santé :

```
curl -i http://127.0.0.1:8000/health
```

```
HTTP/1.1 200 OK
{"status":"ok"}
```

Interprétation : Le code HTTP `200` indique une réponse réussie ; le JSON confirme que notre endpoint de santé répond. `curl` n'est pas une commande Docker, mais elle permet de tester l'application conteneurisée.

9. Confirmer que l'application n'utilise pas root

Si je veux vérifier l'utilisateur Unix à l'intérieur du conteneur Compose en cours d'exécution :

```
docker compose exec secureship id
```

```
uid=10001(appuser) gid=10001(appuser) groups=10001(appuser)
```

À retenir : `exec` exécute une commande dans le conteneur existant. L'UID `10001` confirme que notre application n'est pas lancée sous root (UID 0).

Phase 3 — Comprendre la persistance des données SQLite

10. Retrouver les volumes Docker

Si je veux savoir quels volumes existent et retrouver celui de SecureShip :

```
docker volume ls
```

Résultat attendu : Une liste contenant notamment le volume de notre projet Compose :

```
secureship_secureship-data
```

Ce volume sert à conserver notre base SQLite indépendamment de la durée de vie du conteneur.

11. Consulter les informations du volume persistant

Si je veux connaître les métadonnées du volume de SecureShip :

```
docker volume inspect secureship_secureship-data
```

Résultat attendu : Un document JSON contenant notamment `Name`, `Driver`, `Mountpoint` et `Scope`.
Sur Docker Desktop, le stockage physique du volume peut se trouver dans la machine virtuelle Linux gérée par Docker.

12. Vérifier la présence du fichier SQLite

Si je veux savoir si la base se trouve bien dans le répertoire persistant du conteneur :

```
docker compose exec secureship ls -lah /app/data
```

```
secureship.db
```

Important : Le volume permet la persistance, mais ne remplace pas une sauvegarde. Voir le fichier ne garantit pas qu'une copie de secours existe.

Phase 4 — Gérer SecureShip au quotidien avec Compose

13. Arrêter temporairement SecureShip sans le supprimer

Si j'ai terminé ma séance et que je veux arrêter l'application, tout en conservant le conteneur :

```
docker compose stop
```

Résultat attendu : Le conteneur s'arrête. Il n'est pas supprimé, et le volume SQLite est conservé.

14. Redémarrer un conteneur arrêté

Si j'avais utilisé `docker compose stop` et que je veux relancer le conteneur existant :

```
docker compose start
```

Résultat attendu : Compose redémarre le conteneur existant sans reconstruire l'image. Mes données persistent dans le volume.
Attention : `start` suppose que le conteneur existe encore. Après un `down`, il faut plutôt exécuter `docker compose up -d`.

15. Redémarrer un conteneur en fonctionnement

Si l'application est déjà créée et que je veux redémarrer le processus conteneurisé :

```
docker compose restart
```

Résultat attendu : Compose arrête puis redémarre les conteneurs du projet.
À retenir : `restart` ne reconstruit pas l'image et n'applique pas les changements du code inclus dans une nouvelle image. Pour cela, j'utilise l'étape 17.

16. Lire les journaux quand une opération échoue

Si SecureShip ne démarre pas ou si une requête rencontre une erreur, je veux examiner les messages récents :

```
docker compose logs --tail 50
```

Résultat attendu : Les 50 dernières lignes de journaux du projet Compose.
Si je veux suivre les nouveaux messages en temps réel :

```
docker compose logs -f
```

Pour quitter cet affichage : `Ctrl + C`. Cela arrête le suivi des logs, pas l'application.

17. Appliquer une modification du frontend ou du backend

Si j'ai modifié du code dans VS Code et que je veux appliquer cette nouvelle version à mon environnement Docker :

```
docker compose up -d --build
```

Résultat attendu : Compose reconstruit l'image à partir du `Dockerfile`, puis recrée le conteneur si nécessaire pour lancer le code actualisé.
Le volume nommé est normalement réutilisé : la base SQLite reste indépendante de l'image reconstruite.
Ne pas confondre :
— `docker compose build` : construire l'image, sans démarrer les conteneurs.
— `docker compose up -d` : créer ou démarrer le service ; une image absente peut être construite ou récupérée.
— `docker compose up -d --build` : demander une construction avant le démarrage.
— `docker compose restart` : redémarrer l'existant sans reconstruire le code.

18. Supprimer les conteneurs Compose en préservant SQLite

Si je veux arrêter et supprimer les conteneurs et les réseaux du projet, *sans* supprimer les volumes nommés :

```
docker compose down
```

Résultat attendu : Compose arrête et supprime ses conteneurs et ses réseaux par défaut. Les volumes nommés et les images sont normalement conservés.
Je pourrai recréer et démarrer le service avec :

```
docker compose up -d
```

Ne pas utiliser sur nos données à conserver :

```
docker compose down -v
```

L'option `-v` peut supprimer le volume Compose contenant notre base SQLite. Elle n'est pas appropriée pour un environnement contenant des données à préserver.

Phase 5 — Méthode alternative : Dockerfile sans Docker Compose

Important : Cette phase est un *autre scénario de départ*, pas une suite obligatoire des étapes Compose. Elle s'applique lorsque je dispose du `Dockerfile` mais que je n'utilise pas de fichier Compose. Je dois alors configurer manuellement l'image, les ports, le fichier `.env` et le volume.

19. Construire moi-même l'image avec Dockerfile

Si je travaille sans Docker Compose et que je veux construire l'image de SecureShip :

```
cd ~/Desktop/SecureShip
docker build -t secureship:dev .
```

Résultat attendu : Docker lit le `Dockerfile`, compile React avec Node.js, installe les dépendances Python et crée l'image `secureship:dev`.
Comprendre la commande :
— `build` : construire une image.
— `-t` : définir son nom et son tag.
— `secureship` : nom de l'image.
— `dev` : étiquette choisie, remplaçable par `test`, `v1.0`, etc. Ce tag n'active aucun mode développement.
— `.` : utiliser le dossier courant comme contexte de construction.
Sans tag explicite, Docker utilise `latest`. Notre `Dockerfile` est *multi-stage* : une étape compile le frontend, puis une autre prépare l'image d'exécution FastAPI.

20. Créer et démarrer manuellement un conteneur

Si mon image `secureship:dev` existe et que je veux démarrer l'application sans Compose :

```
docker run -d \\
  --name secureship-dev \\
  -p 127.0.0.1:8000:8000 \\
```

```
  --env-file .env \\
  -v secureship-data:/app/data \\
  secureship:dev
```

Résultat attendu : Docker crée et lance le conteneur `secureship-dev`, puis affiche son identifiant.

À retenir :

— `-d` : démarrer en arrière-plan.
— `--name` : nommer le conteneur.
— `-p` : publier le port local 8000 vers celui du conteneur.
— `--env-file .env` : transmettre la configuration locale nécessaire.
— `-v` : utiliser un volume persistant pour SQLite.

Conditions : `.env` doit être prêt et le port 8000 doit être libre. Si SecureShip tourne déjà avec Compose, je dois d'abord arrêter cet autre conteneur avant d'utiliser le même port.

Attention : Le volume manuel `secureship-data` est différent du volume Compose `secureship_secureship-data`. Les deux scénarios ne partagent pas automatiquement les mêmes expéditions.

21. Vérifier et arrêter mon conteneur manuel

Si je veux inspecter l'utilisateur qui exécute mon application :

```
docker exec secureship-dev id
```

```
uid=10001(appuser) gid=10001(appuser) groups=10001(appuser)
```

Si je veux arrêter ce conteneur sans le supprimer :

```
docker stop secureship-dev
```

Résultat attendu : Le conteneur s'arrête ; son objet et son volume persistent.

22. Redémarrer ou supprimer le conteneur manuel

Si j'ai arrêté `secureship-dev` et que je veux le relancer sans créer un autre conteneur :

```
docker start secureship-dev
```

Si je veux supprimer ce conteneur, je l'arrête d'abord, puis je le supprime :

```
docker stop secureship-dev
docker rm secureship-dev
```

Résultat attendu : L'objet conteneur est supprimé. Le volume nommé `secureship-data` n'est pas supprimé par `docker rm`.

Différence essentielle : `docker run` crée un nouveau conteneur ; `docker start` démarre un conteneur existant ; `docker rm` supprime le conteneur, pas l'image. La suppression d'une image relève de `docker rmi` et peut être refusée si un conteneur l'utilise encore.

Les commandes Docker essentielles au quotidien

Ces commandes servent de fiche de révision. Pour SecureShip, lorsque `Dockerfile` et `docker-compose.yml` sont présents, j'utilise principalement **Docker Compose**. La méthode **Docker classique** reste une alternative si je travaille sans Compose.

1. Docker Compose — Méthode principale pour SecureShip

Ce que je veux faire	Commande
Construire l'image uniquement	`docker compose build`
Créer et démarrer l'application	`docker compose up -d`
Premier démarrage, ou après modification du code	`docker compose up -d --build`
Arrêter l'application temporairement	`docker compose stop`
Redémarrer une application arrêtée	`docker compose start`
Redémarrer les conteneurs	`docker compose restart`
Arrêter et supprimer les conteneurs et réseaux Compose	`docker compose down`

À retenir : La commande `docker compose down` conserve normalement les volumes nommés et les images.
Attention : Ne jamais utiliser `docker compose down -v` sur SecureShip sans avoir explicitement décidé de supprimer les données et vérifié les sauvegardes. Cette commande peut supprimer le volume contenant notre base SQLite.

2. Docker classique — Alternative sans fichier Compose

Ce que je veux faire	Commande
Construire une image	`docker build -t mon-app:dev .`
Créer et démarrer un conteneur	`docker run -d --name mon-conteneur mon-app:dev`
Arrêter un conteneur	`docker stop mon-conteneur`
Démarrer un conteneur déjà créé	`docker start mon-conteneur`
Redémarrer un conteneur	`docker restart mon-conteneur`
Supprimer un conteneur arrêté	`docker rm mon-conteneur`
Forcer la suppression d'un conteneur actif	`docker rm -f mon-conteneur`
Supprimer une image	`docker rmi mon-app:dev`

Remarque : Les noms `mon-app` et `mon-conteneur` sont des exemples. Pour exécuter SecureShip manuellement, il faut également configurer les ports, les variables d'environnement et le volume persistant.


git



Git & GitHub

GUIDE PRATIQUE

SecureShip · 39 situations, commandes et résultats attendus

Objectif : Apprendre et mémoriser les commandes Git à travers les situations réellement rencontrées pendant le développement de SecureShip.
Environnement : MacBook, VS Code, terminal, Git et GitHub.
Méthode : Pour chaque commande, je pars de ce que je veux accomplir, j'exécute la commande, puis j'apprends à interpréter son résultat.

Phase 1 — Initialiser un projet et le connecter à GitHub

Cette phase correspond au début du projet. Les commandes d'initialisation ne doivent pas être répétées dans notre dépôt SecureShip déjà configuré.

1. Me placer dans le dossier de mon projet

Si je veux travailler sur SecureShip et m'assurer que mes prochaines commandes Git seront exécutées depuis le bon dossier :

```
cd ~/Desktop/SecureShip
pwd
```

Résultat attendu :

```
/Users/<utilisateur>/Desktop/SecureShip
```

cd me déplace dans un dossier, tandis que pwd affiche le chemin du dossier dans lequel je me trouve.

2. Vérifier que Git est installé

Si je veux savoir si Git est disponible sur mon MacBook et connaître sa version :

```
git --version
```

Résultat attendu :

```
git version 2.x.x
```

Cela confirme que le terminal reconnaît Git.

3. Initialiser un nouveau projet avec une branche main

Si j'ai créé un nouveau dossier de projet et que je veux commencer à suivre l'évolution de ses fichiers avec Git :

```
git init -b main
```

Résultat attendu : Git crée un dépôt local et initialise une branche appelée main.
Un dossier caché .git/ est créé pour stocker les métadonnées et l'historique du dépôt.
Important : SecureShip possède déjà son dépôt Git. Cette commande sert à comprendre comment nous avons commencé, pas à réinitialiser notre projet actuel.

4. Savoir à quel dépôt GitHub mon projet local est connecté

Si je veux vérifier que mon projet SecureShip est connecté au bon repository GitHub et voir les URL utilisées pour récupérer et publier le code :

```
git remote -v
```

Exemple de résultat :

```
origin  https://github.com/saadkvp03/SecureShip.git (fetch)
origin  https://github.com/saadkvp03/SecureShip.git (push)
```

Je comprends que :
- origin est le nom donné au dépôt distant.
- fetch indique l'adresse utilisée pour récupérer les changements.
- push indique l'adresse utilisée pour publier les commits.

Si je créais un nouveau dépôt GitHub sans connexion distante, je pourrais utiliser :

```
git remote add origin https://github.com/saadkvp03/SecureShip.git
```

Dans SecureShip, cette connexion existe déjà : inutile de l'ajouter à nouveau.

Phase 2 — Comprendre l'état réel de mes fichiers

5. Connaître ma branche actuelle et savoir si j'ai des modifications locales

Si je veux savoir dans quelle branche je travaille, si elle suit une branche distante et si certains fichiers ont été modifiés :

```
git status -sb
```

Exemple 1 : dépôt propre

```
## main...origin/main
```

Cela signifie :

- Je suis sur main.
- Ma branche suit origin/main.
- Git ne signale aucune modification locale.
- Aucun décalage connu n'est indiqué entre les deux références.

Attention : cela ne prouve pas que GitHub n'a reçu aucun nouveau commit depuis notre dernière récupération. Pour obtenir les dernières informations distantes, il faut utiliser git fetch.

Exemple 2 : des modifications existent

```
## main...origin/main
 M README.md
 M app/main.py
?? docs/notes.md
```

Cela signifie que :

- Le README a été modifié.
- app/main.py a été modifié.
- docs/notes.md n'est pas encore suivi par Git (untracked).

6. Distinguer un fichier modifié, préparé et non suivi

Si je veux afficher uniquement les fichiers concernés par des modifications et comprendre leur état sans lire la version longue de git status :

```
git status --short
```

Exemple de résultat :

```
 M README.md
M  app/main.py
A  docs/rapport.md
?? docs/assets/image.png
```

Comment lire les deux premières colonnes :

Symbole	Ce que je comprends
␣M	Fichier suivi, modifié dans mon dossier, mais pas encore préparé
M␣	Modification déjà préparée pour le prochain commit
A␣	Nouveau fichier ajouté à la zone de préparation
??	Fichier non suivi par Git
MM	Fichier préparé, puis modifié à nouveau après sa préparation

À retenir : un fichier tracked est connu de Git. Un fichier untracked existe localement, mais Git ne l'a pas encore ajouté à son suivi.

7. Voir tous les nouveaux fichiers d'un dossier

Si Git m'affiche seulement :

```
?? docs/assets/
```

et que je veux voir exactement quelles images ou quels fichiers existent à l'intérieur du dossier :

```
git status --short --untracked-files=all
```

Exemple de résultat :

```
?? docs/assets/app-login.png
?? docs/assets/app-dashboard.png
?? docs/assets/workflow-global.png
```

Cette commande est particulièrement utile avant de publier un nouveau dossier.
Dans SecureShip : nous l'avons utilisée pour vérifier individuellement les 14 images préparées pour notre README.

8. Savoir quels fichiers sont déjà suivis par Git

Si je veux vérifier si un fichier particulier fait déjà partie des fichiers suivis :

```
git ls-files README.md
```

Résultat attendu :

```
README.md
```

Si cette commande ne renvoie rien, le chemin indiqué n'est pas présent parmi les fichiers suivis dans l'index Git.
Pour afficher tous les fichiers suivis :

```
git ls-files
```

9. Vérifier pourquoi un fichier sensible n'est pas ajouté

Si je veux savoir si mon fichier .env est exclu par .gitignore et quelle règle provoque cette exclusion :

```
git check-ignore -v .env
```

Exemple de résultat :

```
.gitignore:8:.env    .env
```

Cela signifie qu'une règle de .gitignore correspond au fichier .env.
Attention : .gitignore n'arrête pas automatiquement le suivi d'un fichier déjà enregistré dans Git. Il faut donc toujours éviter d'ajouter des secrets dès le départ.

Phase 3 — Examiner mes modifications avant de les enregistrer

10. Voir ce que j'ai réellement changé dans un fichier

Si j'ai modifié du code dans VS Code et que je veux comparer ma version actuelle avec la dernière version connue de Git, avant de préparer les modifications :

```
git diff
```

Résultat attendu : Git présente les parties modifiées.
- Une ligne précédée de - représente du contenu retiré.
- Une ligne précédée de + représente du contenu ajouté.

Si je veux examiner uniquement les modifications du backend :

```
git diff -- app/main.py
```

À retenir : git diff, sans option, montre principalement les modifications qui ne sont pas encore préparées pour le commit.

11. Simuler l'ajout de fichiers avant de les préparer

Si je veux savoir quels fichiers seraient ajoutés à mon prochain commit en utilisant git add ., sans encore les préparer réellement :

```
git add --dry-run .
```

Résultat attendu : Git liste les chemins qu'il ajouterait ou actualiserait.
Nous avons utilisé cette vérification pour éviter d'inclure des fichiers inutiles.
Dans SecureShip, les dossiers et fichiers tels que .venv/, frontend/node_modules/, frontend/dist/, .env et les bases *.db doivent rester hors de nos commits.

12. Préparer des modifications pour mon prochain commit

Si mes modifications sont terminées et que je veux sélectionner les fichiers qui doivent figurer dans le prochain enregistrement Git :

```
git add README.md
```

Résultat attendu : Aucun message en cas de succès. La version actuelle du README est placée dans la staging area.
Si je veux sélectionner plusieurs fichiers précis :

```
git add README.md docs/03-security-findings.md
```

Si je souhaite préparer toutes les modifications pertinentes situées dans mon dossier courant :

```
git add .
```

Attention : git add . peut préparer plus de fichiers que prévu. Je dois toujours vérifier le résultat avant de créer le commit.

13. Vérifier que mes fichiers ont bien été préparés

Si j'ai exécuté git add et que je veux savoir quels fichiers figurent maintenant dans la zone de préparation :

```
git status -sb
```

Exemple de résultat :

```
## main...origin/main
M  README.md
M  docs/03-security-findings.md
A  docs/assets/app-login.png
```

Les lettres dans la première colonne indiquent que ces changements sont préparés.

14. Voir combien de fichiers et de lignes mon commit va contenir

Si j'ai préparé plusieurs fichiers et que je veux obtenir un résumé avant de les enregistrer :

```
git diff --cached --stat
```

Exemple basé sur notre dernier travail documentaire :
README.md | 127 ++++++++++++++++++—

```
docs/01-scope-architecture.md | ...
docs/05-final-report.md    | ...
docs/assets/app-login.png  | Bin 0 -> 42306 bytes
...
22 files changed, 1374 insertions(+), 516 deletions(-)
```

Je comprends que Git va enregistrer :

- Des fichiers Markdown modifiés.
- Des images nouvellement ajoutées.
- Un certain nombre de lignes ajoutées et supprimées.

Important : les nombres de lignes dans ce résumé ne sont pas des tests de qualité du code. Ils servent à comprendre le volume du changement.

15. Examiner précisément les modifications déjà préparées

Si je veux lire les changements qui seront réellement inclus dans mon prochain commit, et non simplement les changements de mon dossier de travail :

```
git diff --cached
```

Résultat attendu : Git affiche les différences présentes dans la staging area.
La distinction essentielle est :

- git diff

$rightarrow$ modifications non préparées.

- git diff –cached

$rightarrow$ modifications préparées pour le prochain commit.

16. Vérifier les erreurs d'espacement avant le commit

Si j'ai préparé des fichiers Markdown, YAML ou Python et que je veux détecter les erreurs d'espacement et certains marqueurs de conflit dans les modifications qui seront enregistrées :

```
git diff --cached --check
```

Résultat attendu si aucune anomalie n'est détectée : le terminal n'affiche rien et la commande se termine avec succès.
Exemple de problème que nous avons réellement rencontré :

```
docs/01-scope-architecture.md:4: trailing whitespace.
```

Cela signifiait que Git avait détecté des espaces en fin de ligne.
Dans nos documents Markdown, certains doubles espaces servaient aux retours à la ligne. Nous les avons remplacés par une syntaxe Markdown équivalente, puis ajouté les fichiers corrigés à la staging area.
Après cela, nous avons relancé la commande : aucun avertissement.
Ce contrôle ne vérifie pas toute la qualité du code ; il détecte principalement des problèmes de formatage.

17. Retirer un fichier de mon prochain commit sans supprimer mon travail

Si j'ai préparé un fichier par erreur et que je veux l'enlever de la staging area, tout en conservant mes modifications dans VS Code :

```
git restore --staged README.md
```

Résultat attendu : Le README ne fait plus partie des modifications préparées, mais les modifications locales restent présentes.
Attention à ne pas confondre avec :

```
git restore README.md
```

Cette deuxième forme peut supprimer les modifications non préparées du fichier. Je ne dois pas l'exécuter sans comprendre les conséquences.

Phase 4 — Créer un commit et comprendre l'historique Git

18. Enregistrer mes modifications localement

Si j'ai terminé mes modifications, que mes tests sont satisfaisants et que j'ai vérifié les fichiers préparés, je veux créer un point identifiable dans l'historique Git :

```
git commit -m "docs: update DevSecOps security reports and visual evidence"
```

Exemple de résultat :

```
[main d3fa9c0] docs: update DevSecOps security reports and visual evidence
22 files changed, 1374 insertions(+), 516 deletions(-)
```

C'est le commit de notre dernière mise à jour documentaire SecureShip.
Je comprends que :

- main est la branche sur laquelle le commit a été créé.
- d3fa9c0 est l'identifiant abrégé du commit.
- Le message explique la modification.
- Les 22 fichiers préparés ont été enregistrés dans un nouvel instantané.

Important : git commit ne publie rien sur GitHub. Il enregistre les modifications localement.

19. Retrouver mes derniers commits

Si je veux voir les derniers changements enregistrés dans mon projet, dans leur ordre historique, avec leurs identifiants et leurs messages :

```
git log --oneline -10
```

Exemple de résultat :

```
d3fa9c0 docs: update DevSecOps security reports and visual evidence
15260ab Replace old image link with new one in README
71afa68 Merge pull request #2 from saadkvp03/feature/security-hardening
...
```

Je retrouve ainsi les étapes importantes de SecureShip.

20. Visualiser les différentes branches et leurs fusions

Si je veux comprendre graphiquement l'historique Git et observer les branches de développement ainsi que leurs merges :

```
git log --oneline --graph --decorate --all
```

Résultat attendu : Git dessine un historique textuel à l'aide de symboles tels que *, |, / et \.
Les étiquettes peuvent également indiquer les positions de main, origin/main et des branches feature/.
Si Git affiche (END), je peux appuyer sur q pour quitter la visualisation.

21. Voir les fichiers inclus dans mon dernier commit

Si je veux vérifier ce que mon dernier commit a réellement enregistré, sans lire toutes les modifications ligne par ligne :

```
git show --stat --oneline HEAD
```

Résultat attendu : Le nom du dernier commit, son identifiant et la liste des fichiers modifiés avec leurs statistiques.
HEAD représente la position actuelle de mon historique Git.

Phase 5 — Travailler avec plusieurs branches Git

22. Connaître toutes mes branches locales et identifier la branche active

Si je veux savoir quelles branches existent dans mon dépôt et identifier celle sur laquelle je travaille actuellement :

```
git branch
```

Exemple de résultat :

```
  feature/security-hardening
* main
```

Le symbole * désigne la branche actuelle.

23. Voir les branches, les derniers commits et les connexions distantes

Si je veux savoir quelles branches possèdent une correspondance sur GitHub et voir leur position par rapport à cette correspondance :

```
git branch -vv
```

Résultat attendu : Chaque branche peut être accompagnée de son commit courant, de sa branche distante suivie et d'une indication de décalage comme ahead ou behind.
Par exemple, ahead 1 signifie que la branche locale contient un commit supplémentaire par rapport à sa référence distante connue.
Cette information dépend de la dernière récupération des références distantes.

24. Créer une branche pour développer une nouvelle fonctionnalité

Si mon application fonctionne correctement sur main et que je veux travailler sur une nouvelle fonctionnalité sans modifier directement cette branche :

```
git switch -c feature/security-hardening
```

Résultat attendu :

```
Switched to a new branch 'feature/security-hardening'
```

Git crée la branche et m'y place immédiatement.
Dans SecureShip : nous avons utilisé cette approche pour développer les améliorations de sécurité avant de les proposer dans la Pull Request #2.
Cette commande sert à illustrer notre étape historique. Une branche portant ce nom peut déjà exister dans ton dépôt.

25. Passer d'une branche à une autre

Si je travaille sur une branche de développement et que je veux revenir à la branche principale :

```
git switch main
```

Résultat attendu :

```
Switched to branch 'main'
```

Si je veux revenir sur une branche existante :

```
git switch feature/security-hardening
```

Différence à mémoriser :

- git switch main : aller sur une branche existante.
- git switch -c feature/... : créer une nouvelle branche puis y aller.

Git peut refuser de changer de branche si des modifications non enregistrées risquent d'être écrasées. Je dois alors examiner git status avant toute autre action.

Phase 6 — Publier mon travail sur GitHub

26. Publier une nouvelle branche sur GitHub

Si j'ai développé et enregistré des commits sur une nouvelle branche locale et que je veux rendre cette branche disponible sur GitHub :

```
git push -u origin feature/security-hardening
```

Résultat attendu : Git envoie les commits nécessaires et crée ou met à jour la branche distante correspondante.
L'option -u configure le suivi distant (upstream), ce qui facilite les prochains échanges avec GitHub.
Dans SecureShip : nous avons publié la branche de sécurité pour pouvoir ouvrir notre Pull Request #2.

27. Publier les nouveaux commits de ma branche actuelle

Si ma branche possède déjà une connexion distante et que je veux envoyer mes derniers commits sur GitHub :

```
git push
```

Résultat attendu : Les commits locaux qui ne sont pas encore présents sur la branche distante sont publiés.
Si je veux explicitement publier main :

```
git push origin main
```

Dans notre dernière séance : c'est cette commande qui a publié le commit documentaire d3fa9c0.

28. Vérifier si mes commits sont déjà publiés

Si je veux savoir si ma branche contient encore des commits locaux qui n'ont pas été envoyés sur GitHub :

```
git status -sb
```

Exemple :

```
## main...origin/main [ahead 1]
```

Cela signifie que ma branche locale possède un commit d'avance sur sa référence distante connue.
Après un push réussi, le résultat peut redevenir :

```
## main...origin/main
```

Cela indique qu'aucun décalage connu n'est signalé.

Phase 7 — Utiliser les Pull Requests et GitHub Actions

29. Proposer l'intégration de mes modifications dans main

Si j'ai terminé une fonctionnalité sur feature/security-hardening et que je veux proposer son intégration dans main sans fusionner immédiatement les changements :

- Je publie la branche sur GitHub.
- J'ouvre le dépôt dans mon navigateur.
- Je sélectionne Compare & pull request.
- Je choisis la branche source feature/security-hardening et la branche de destination main.
- Je crée la Pull Request.

Résultat attendu : GitHub crée une page de Pull Request où je peux consulter les commits, les fichiers modifiés, les commentaires et les résultats CI.
Une Pull Request est une proposition de fusion ; ce n'est pas encore un merge.

30. Vérifier que ma Pull Request ne casse pas l'application

Si j'ai ouvert une Pull Request et que je veux savoir si mes modifications passent les tests et les contrôles de sécurité avant d'accepter la fusion :
Je consulte la section Checks sur GitHub.
Dans SecureShip, le pipeline actuel exécute sept jobs :

Job	Ce que je vérifie
test	Les tests Python passent
frontend	Le frontend React/Vite compile
docker	L'image se construit, le conteneur fonctionne et Trivy applique sa politique
semgrep	Le scan SAST passe
gitleaks	Aucun secret n'est détecté selon les règles du scanner
sca	Les audits Python et npm de production passent
checkov	Les configurations Dockerfile et GitHub Actions passent leurs contrôles

Résultat attendu : GitHub affiche le succès ou l'échec de chaque job.
Dans notre Pull Request #2, nous avons observé 14 checks réussis au total, issus des exécutions de push et de Pull Request.
Attention : un pipeline vert ne garantit pas l'absence de vulnérabilités. Il indique que les contrôles configurés ont réussi.

31. Fusionner une Pull Request dans main

Si j'ai examiné les fichiers, accepté les modifications et vérifié les résultats nécessaires, je veux intégrer la branche de développement à main :
Sur GitHub :

```
Merge pull request → Confirm merge
```

Résultat attendu : GitHub crée l'intégration sur main et marque la Pull Request comme fusionnée.
Dans SecureShip : la Pull Request #2 a été fusionnée avec le commit :

```
71afa68
```

La fusion a intégré notre travail de sécurité dans la branche principale.
Attention : les conditions de revue et de contrôle obligatoires dépendent des paramètres du dépôt GitHub. Un pipeline vert ne signifie pas nécessairement que GitHub interdit toutes les fusions non vérifiées.

Phase 8 — Synchroniser mon MacBook avec GitHub

32. Récupérer les informations récentes de GitHub sans changer mes fichiers locaux

Si je veux savoir si GitHub contient de nouveaux commits, mais que je ne veux pas encore les intégrer dans ma branche de travail :

```
git fetch origin
```

Résultat attendu : Git met à jour les références distantes connues, par exemple origin/main, sans modifier directement mes fichiers de travail.
Je peux ensuite exécuter :

```
git status -sb
```

Exemple :

```
## main...origin/main [behind 1]
```

Cela signifie que GitHub possède un commit supplémentaire que ma branche locale n'a pas encore intégré.

33. Mettre mon main local à jour après un merge sur GitHub

Si j'ai fusionné une Pull Request sur GitHub et que je veux récupérer la nouvelle version de main sur mon MacBook, en refusant une fusion automatique non linéaire :

```
git pull --ff-only origin main
```

Exemple de résultat :

```
Updating 71afa68..15260ab
Fast-forward
README.md | 209 +++++++++
```

Cela signifie que Git a avancé ma branche locale vers la nouvelle version sans créer de commit de fusion supplémentaire.
Dans SecureShip : nous avons utilisé cette commande après les modifications du README effectuées directement sur GitHub.
Si ma branche contient des changements incompatibles ou divergents, –ff-only peut refuser la mise à jour. C'est une protection utile : je dois comprendre la divergence avant de décider comment la résoudre.

34. Vérifier que mon ordinateur et GitHub sont synchronisés

Si je viens d'exécuter git pull ou git push et que je veux confirmer qu'aucun décalage connu ni changement local n'est signalé :

```
git status -sb
```

Résultat attendu :

```
## main...origin/main
```

Notre dépôt local est alors propre et synchronisé avec la référence distante connue.

Les commandes Git à mémoriser absolument

Si je veux…	Commande
Connaître ma branche et les modifications locales	`git status -sb`
Voir seulement les fichiers modifiés ou nouveaux	`git status --short`
Voir les fichiers à l'intérieur des dossiers non suivis	`git status --short --untracked-files=all`
Voir mes branches	`git branch`
Voir les branches et leur suivi distant	`git branch -vv`
Voir les changements non préparés	`git diff`
Simuler les fichiers que j'ajouterais	`git add --dry-run .`
Préparer un fichier précis	`git add README.md`
Préparer les changements du dossier courant	`git add .`
Voir les modifications préparées	`git diff --cached`
Voir un résumé des modifications préparées	`git diff --cached --stat`
Détecter les erreurs d'espacement avant commit	`git diff --cached --check`
Retirer un fichier du staging sans perdre mes modifications	`git restore --staged README.md`
Enregistrer un commit local	`git commit -m "message"`
Voir les derniers commits	`git log --oneline -10`
Voir le contenu résumé du dernier commit	`git show --stat --oneline HEAD`
Créer une branche et m'y placer	`git switch -c feature/nom`
Revenir sur main	`git switch main`
Publier une nouvelle branche	`git push -u origin feature/nom`
Publier main sur GitHub	`git push origin main`
Récupérer les références distantes sans intégrer les commits	`git fetch origin`
Récupérer la nouvelle version de main	`git pull --ff-only origin main`
Vérifier le dépôt GitHub connecté	`git remote -v`

Le workflow Git professionnel à retenir

Lorsque je veux développer une nouvelle fonctionnalité dans SecureShip, voici la démarche que je dois comprendre et reproduire, en adaptant le nom de branche et les tests au travail concerné.

1. Si je veux commencer depuis la dernière version de main :

```
git switch main
git pull --ff-only origin main
```

2. Si je veux isoler mes modifications de la branche principale :

```
git switch -c feature/nouvelle-fonctionnalite
```

3. Si je veux voir ce que j'ai modifié après avoir travaillé dans VS Code :

```
git status -sb
git diff
```

4. Si je veux préparer et vérifier les fichiers :

```
git add .
git diff --cached --stat
git diff --cached --check
```

5. Si je veux enregistrer mes changements localement :

```
git commit -m "feat: describe my change"
```

6. Si je veux envoyer la branche sur GitHub :

```
git push -u origin feature/nouvelle-fonctionnalite
```

7. Si je veux proposer une intégration contrôlée :

Je crée une Pull Request vers main, j'examine les modifications et les résultats GitHub Actions, puis je fusionne lorsque les conditions du projet sont satisfaites.

8. Si je veux récupérer la version fusionnée sur mon MacBook :

```
git switch main
git pull --ff-only origin main
git status -sb
```

Les cinq concepts que je ne dois jamais confondre

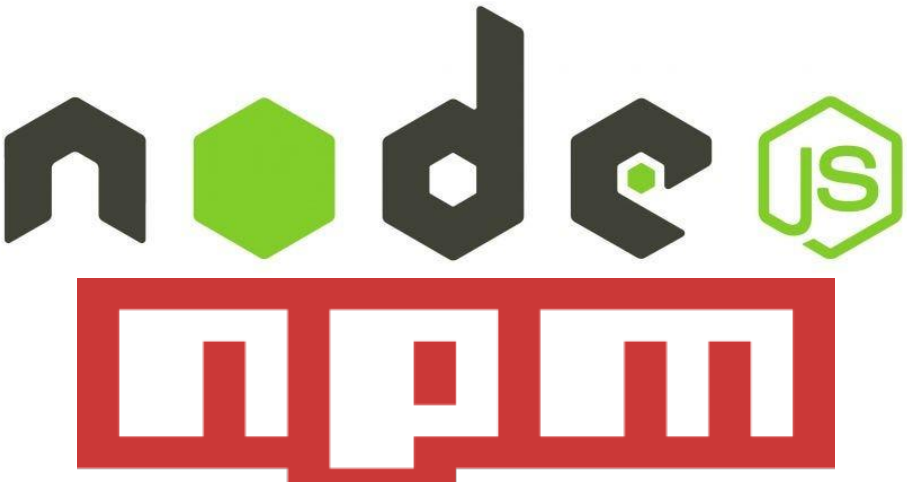
Concept	Ce que je dois retenir
Working Directory	Les fichiers sur lesquels je travaille dans VS Code
Staging Area	Les modifications que j'ai sélectionnées avec git add
Commit	Un instantané enregistré dans l'historique Git local
Push	L'envoi des commits locaux vers GitHub
Pull Request / Merge	Une proposition de fusion, puis l'intégration effective des changements

Conclusion

Dans SecureShip, Git nous a permis de conserver l'historique de développement, d'isoler les changements importants et de suivre nos corrections de sécurité.

GitHub nous a permis de collaborer à travers les Pull Requests et d'exécuter automatiquement les tests et analyses DevSecOps après les pushes.

Pour un ingénieur cybersécurité, Git n'est donc pas seulement un outil de sauvegarde de code : c'est également un outil de traçabilité, de vérification et de contrôle du cycle de développement logiciel.


node
JS
npm

Node.js & npm

GUIDE PRATIQUE

SecureShip · 20 situations, commandes et résultats attendus

Objectif : Comprendre comment installer, démarrer, modifier, tester et compiler le frontend de SecureShip à l'aide de Node.js, npm et Vite.
Technologies utilisées : React, TypeScript, Vite, Tailwind CSS et npm.

Phase 1 — Vérifier Node.js et npm

1. Vérifier si Node.js est installé sur mon MacBook

Si je veux savoir si Node.js est installé et connaître sa version :

```
node --version
```

Exemple de résultat :

```
v20.x.x
```

Cela confirme que Node.js est disponible. Notre projet utilise Node.js 20 dans GitHub Actions.

2. Vérifier que npm est installé

Si je veux savoir si le gestionnaire de paquets npm est disponible sur ma machine :

```
npm --version
```

Résultat attendu : Un numéro de version npm, par exemple 10.x.x.
À retenir :
— Node.js permet d'exécuter les outils JavaScript.
— npm permet d'installer et de gérer les dépendances du projet.
— Vite est l'outil qui démarre le serveur de développement et construit notre frontend.

3. Me placer dans le dossier de notre frontend

Si je veux travailler uniquement sur la partie React de SecureShip :

```
cd ~/Desktop/SecureShip/frontend
```

Pour vérifier que je suis au bon endroit :

```
pwd
```

Résultat attendu :

```
/Users/<utilisateur>/Desktop/SecureShip/frontend
```

Ce dossier contient notamment package.json, package-lock.json, vite.config.ts et le répertoire src/.

Phase 2 — Installer les dépendances React

4. Installer les bibliothèques nécessaires au frontend

Si je viens de télécharger le projet pour la première fois et que je veux installer React, Vite, TypeScript et toutes les dépendances déclarées dans package.json :

```
npm install
```

Résultat attendu :
— npm télécharge les dépendances nécessaires.
— Le dossier node_modules/ est créé.
— Le fichier package-lock.json est créé ou actualisé si nécessaire.
— npm affiche un résumé de l'installation et éventuellement des alertes de sécurité.
Dans SecureShip, node_modules/ est exclu de Git, car les dépendances peuvent être réinstallées.

5. Réinstaller exactement les dépendances définies dans package-lock.json

Si je veux installer les dépendances de manière reproductible, en respectant le fichier package-lock.json existant :

```
npm ci
```

Résultat attendu : npm installe les dépendances à partir du lockfile, sans le modifier. Il recrée node_modules/ et échoue si package.json et package-lock.json ne sont pas cohérents.
Différence essentielle :
— npm install : permet d'installer et éventuellement de mettre à jour le lockfile.
— npm ci : privilégie une installation reproductible à partir du lockfile.
Dans GitHub Actions, SecureShip utilise npm ci lorsque package-lock.json existe.

Phase 3 — Démarrer le frontend React

6. Démarrer le serveur de développement React

Si j'ai installé les dépendances et que je veux voir mon interface React dans le navigateur pendant que je travaille dans VS Code :

```
npm run dev
```

Exemple de résultat :

```
VITE v5.x.x ready
Local: http://localhost:5173/app/
```

Je peux maintenant ouvrir :

```
http://localhost:5173/app/
```

C'est le serveur de développement Vite, et non le serveur Python FastAPI.
À retenir : npm run dev exécute le script dev défini dans package.json, qui démarre Vite.

7. Comprendre pourquoi le frontend a besoin du backend

Si j'ai démarré uniquement React et que je vois la page de connexion, mais que les opérations d'authentification ou de gestion des expéditions ne fonctionnent pas, je dois vérifier que le backend FastAPI est également démarré.
Notre frontend communique avec le backend pour :
— Authentifier l'opérateur.
— Récupérer la liste des expéditions.
— Créer et modifier les expéditions.
— Consulter les statistiques du dashboard.
Le frontend ne remplace pas le backend.
Dans SecureShip, Vite transmet les requêtes /auth, /shipments, /stats et /health vers FastAPI sur 127.0.0.1 :8000.

8. Démarrer le backend FastAPI et le frontend React ensemble

Si je veux développer l'application complète, avec un frontend réactif et un backend disponible, je dois utiliser deux terminaux distincts.
Terminal 1 — Démarrer FastAPI
Depuis la racine du projet :

```
cd ~/Desktop/SecureShip
source .venv/bin/activate
python -m uvicorn app.main:app --reload --env-file .env
```

Résultat attendu :

```
Uvicorn running on http://127.0.0.1:8000
```

Le backend est démarré, avec la configuration d'authentification locale chargée depuis .env.
Terminal 2 — Démarrer React/Vite

```
cd ~/Desktop/SecureShip/frontend
npm run dev
```

Résultat attendu :

```
Local: http://localhost:5173/app/
```

Maintenant, je peux ouvrir :

Adresse	Ce que je vais voir
`http://localhost:5173/app/`	Interface React en développement
`http://127.0.0.1:8000/health`	Réponse de santé du backend
`http://127.0.0.1:8000/docs`	Documentation Swagger de FastAPI

Important : le fichier .env doit déjà exister. Je ne dois pas relancer scripts/setup_auth.py si mes identifiants sont déjà configurés.

9. Voir immédiatement mes modifications React sans reconstruire l'application

Si je travaille dans un fichier React ou CSS et que je veux voir les changements dans mon navigateur sans reconstruire manuellement l'image Docker :
Je garde cette commande active :

```
npm run dev
```

Puis je modifie mes fichiers dans VS Code.
Résultat attendu : Vite actualise automatiquement l'interface dans le navigateur grâce au Hot Module Replacement (HMR), lorsque le changement est compatible.
C'est pourquoi nous utilisons Vite pendant le développement du frontend.

10. Arrêter le serveur de développement React

Si j'ai terminé de développer le frontend et que je veux arrêter Vite :
Dans le terminal où npm run dev fonctionne, j'appuie sur :

```
Ctrl + C
```

Résultat attendu : Le processus Vite s'arrête et le terminal redevient disponible.
Cette action n'arrête pas automatiquement FastAPI s'il fonctionne dans un autre terminal.

Phase 4 — Vérifier et compiler le frontend

11. Vérifier les erreurs TypeScript sans compiler le frontend

Si j'ai modifié plusieurs composants React et que je veux savoir si TypeScript détecte des problèmes de types avant de construire l'application :

```
npm run typecheck
```

Résultat attendu : Si le contrôle réussit, la commande se termine sans erreur.
Si TypeScript détecte un problème, il affiche le fichier concerné, la ligne et un message décrivant l'erreur.
Dans SecureShip, ce script exécute :

```
tsc --noEmit
```

L'option --noEmit vérifie le code sans produire les fichiers JavaScript compilés.

12. Construire la version compilée du frontend React

Si mon travail sur React est terminé et que je veux produire les fichiers statiques destinés à être servis par FastAPI ou intégrés dans Docker :

```
npm run build
```

Résultat attendu :
— TypeScript vérifie le code.
— Vite compile et optimise le frontend.
— Un dossier frontend/dist/ est créé ou actualisé.
Notre script build exécute :

```
tsc --noEmit && vite build
```

Le symbole && signifie que la compilation Vite est lancée uniquement si la vérification TypeScript réussit.
Différence importante :
— npm run dev : lancer un serveur pour développer.
— npm run typecheck : vérifier TypeScript sans produire de build.
— npm run build : vérifier TypeScript et produire la version compilée.

13. Vérifier que le dossier de compilation a été créé

Si je viens d'exécuter npm run build et que je veux voir les fichiers générés :

```
ls -lah dist
```

Résultat attendu : Un dossier contenant notamment :

```
index.html
assets/
```

Ces fichiers représentent la version compilée du frontend.
Le répertoire frontend/dist/ est exclu de Git, car il peut être régénéré à partir du code source.

Phase 5 — Exécuter le frontend compilé avec FastAPI

14. Faire fonctionner React et FastAPI sur le même port

Si j'ai terminé le développement du frontend et que je veux tester l'application complète dans une configuration proche de notre déploiement Docker, sans utiliser le serveur Vite :
Étape 1 — Compiler React
Depuis le dossier frontend :

```
npm run build
```

Étape 2 — Revenir à la racine du projet

```
cd ..
```

Étape 3 — Démarrer FastAPI

```
source .venv/bin/activate
python -m uvicorn app.main:app --env-file .env
```

Résultat attendu : FastAPI sert à la fois :
— Le backend REST.
— Le frontend React compilé dans frontend/dist/.
Je peux ouvrir :

```
http://127.0.0.1:8000/app
```

Dans ce mode, nous n'avons plus besoin d'exécuter npm run dev pour afficher le frontend compilé.
À mémoriser :

Mode	Frontend	Backend
Développement	Vite, port 5173	FastAPI, port 8000
Application compilée	React servi par FastAPI	FastAPI, port 8000
Docker Compose	React compilé dans l'image Docker	FastAPI dans le même conteneur

15. Voir le frontend compilé directement dans Docker

Si je veux exécuter la version conteneurisée de SecureShip après avoir modifié le frontend :
Depuis la racine du projet :

```
docker compose up -d --build
```

Résultat attendu :
— Docker reconstruit le frontend React.
— Il intègre les fichiers compilés dans l'image Python.
— Le conteneur SecureShip démarre avec la nouvelle interface.
Je peux ensuite ouvrir :

```
http://127.0.0.1:8000/app
```

Ici, c'est Docker qui gère la construction et le lancement de notre application.
Le volume SQLite existant est normalement conservé.

Phase 6 — Examiner les dépendances et les vulnérabilités npm

16. Voir les dépendances principales installées dans mon frontend

Si je veux connaître les paquets installés directement par mon application, avec leurs versions :

```
npm ls --depth=0
```

Résultat attendu : Une liste contenant notamment React, React DOM, TypeScript, Vite, Tailwind CSS et les autres bibliothèques du frontend.
Cette commande permet de voir les dépendances installées, sans afficher toute leur arborescence transitive.

17. Vérifier si les dépendances npm présentent des vulnérabilités connues

Si je veux analyser les dépendances de mon frontend pour voir si npm connaît des problèmes de sécurité qui les affectent :

```
npm audit
```

Résultat attendu : npm affiche un rapport comprenant éventuellement des vulnérabilités, leur niveau de sévérité et les paquets concernés.
Résultat historique de SecureShip, en octobre 2026 :

```
9 vulnerabilities
6 high
3 moderate
```

Ces vulnérabilités concernaient des dépendances de développement ou de compilation dans notre inventaire testé.
Ce résultat peut évoluer lors de nouvelles installations ou lorsque la base d'avis de sécurité est mise à jour.

18. Vérifier uniquement les dépendances de production

Si je veux savoir si les dépendances utilisées par le frontend en production présentent des vulnérabilités connues, sans inclure les dépendances exclusivement utilisées pendant le développement :

```
npm audit --omit=dev
```

Résultat observé pendant notre validation SecureShip :

```
found 0 vulnerabilities
```

Cela signifie qu'aucune vulnérabilité connue n'a été signalée dans le périmètre de cet audit.
Attention : cela ne signifie pas que les neuf vulnérabilités du scan complet ont été corrigées. Les dépendances de développement peuvent également représenter un risque pour la chaîne de compilation.
C'est cette commande que nous utilisons actuellement dans le job sca de GitHub Actions.

19. Comprendre quelles mises à jour de dépendances sont disponibles

Si je veux identifier les paquets pour lesquels npm connaît une version plus récente :

```
npm outdated
```

Résultat attendu : Un tableau avec notamment :

```
Package   Current   Wanted   Latest
```

Je peux comprendre :
— Current : version actuellement installée.
— Wanted : version la plus récente compatible avec la plage déclarée dans package.json.
— Latest : dernière version publiée selon npm.
Attention : une version plus récente n'est pas automatiquement compatible avec notre projet.
Dans SecureShip, nous avons volontairement évité d'appliquer aveuglément npm audit fix --force, qui pourrait introduire des mises à jour incompatibles avec notre frontend.

Phase 7 — Comprendre les scripts npm

20. Voir les commandes npm définies dans notre projet

Si j'oublie les scripts disponibles dans SecureShip et que je veux les retrouver sans ouvrir manuellement package.json :

```
npm run
```

Résultat attendu : npm affiche les scripts disponibles :

```
dev
typecheck
build
preview
```

Ces scripts correspondent à :

Script	Ce que je souhaite obtenir
`npm run dev`	Démarrer le frontend React en développement
`npm run typecheck`	Vérifier les erreurs TypeScript
`npm run build`	Générer le frontend compilé
`npm run preview`	Prévisualiser localement les fichiers déjà compilés

Attention : npm run preview est un outil de prévisualisation statique de Vite. Dans la configuration actuelle de SecureShip, il ne configure pas le même proxy API que npm run dev. Pour tester une connexion et les fonctionnalités complètes, je dois utiliser le mode développement avec FastAPI ou laisser FastAPI servir le frontend compilé.

Résumé — Les commandes Node.js et npm à mémoriser

Si je veux…	Commande
Connaître ma version Node.js	`node --version`
Connaître ma version npm	`npm --version`
Installer les dépendances du frontend	`npm install`
Réinstaller les dépendances du lockfile	`npm ci`
Lancer React en développement	`npm run dev`
Vérifier TypeScript	`npm run typecheck`
Compiler le frontend	`npm run build`
Voir les fichiers générés	`ls -lah dist`
Voir les scripts disponibles	`npm run`
Voir les dépendances principales installées	`npm ls --depth=0`
Analyser toutes les dépendances npm	`npm audit`
Analyser les dépendances de production	`npm audit --omit=dev`
Identifier les versions plus récentes	`npm outdated`
Démarrer l'application compilée avec FastAPI	`python -m uvicorn app.main:app --env-file .env (depuis la racine)`
Reconstruire et lancer SecureShip avec Docker	`docker compose up -d --build (depuis la racine)`

Le workflow React à mémoriser dans SecureShip

Lorsque je veux modifier l'interface de SecureShip :

1. Je démarre FastAPI dans un premier terminal avec la configuration locale .env.
2. Je démarre Vite dans un deuxième terminal avec npm run dev.
3. Je développe mes composants React dans VS Code et observe les changements sur http ://localhost :5173/app/.
4. Je vérifie TypeScript avec npm run typecheck.
5. Je construis la version finale du frontend avec npm run build.
6. Je vérifie éventuellement les dépendances avec npm audit --omit=dev, en consultant également les alertes de l'audit complet.
7. Je teste l'application compilée sur FastAPI ou dans Docker.
8. Je prépare mon commit Git après les validations nécessaires.

Ce que je dois retenir : npm installe les dépendances et exécute les scripts du frontend. Vite facilite le développement et la compilation. React fournit l'interface utilisateur, tandis que FastAPI gère l'authentification, la logique métier et la base de données.
Dans SecureShip, cette séparation permet de travailler rapidement sur l'interface, puis d'intégrer les fichiers React compilés dans notre image Docker pour exécuter l'application complète.

Security Hardening

Outils d’analyse et de validation DevSecOps

Semgrep · Gitleaks · Trivy

pip-audit · npm audit

Checkov · Actionlint / ShellCheck

Security Hardening

GUIDE PRATIQUE

SecureShip · 42 situations, commandes et résultats attendus

Objectif : Apprendre à utiliser Semgrep, Gitleaks, Trivy, pip-audit, npm audit, Checkov et Actionlint à partir de situations concrètes rencontrées pendant le développement de SecureShip.
Méthode d'apprentissage : Pour chaque situation, je pars du besoin : « Si je veux… », puis je retrouve la commande et j'apprends à interpréter sa sortie.
Environnement : MacBook, terminal VS Code, Python, React, Docker et GitHub Actions.
Important : Les résultats chiffrés correspondent à nos validations du 18 octobre 2026. Un nouveau scan peut produire d'autres résultats selon les versions, les dépendances et les bases de vulnérabilités.
Sauf indication contraire, les commandes s'exécutent depuis :

```
cd ~/Desktop/SecureShip
```

Partie 1 — Semgrep : Static Application Security Testing (SAST)

1. Vérifier si Semgrep est installé sur mon MacBook

Si je veux savoir si Semgrep est disponible sur mon MacBook et connaître la version que j'utilise :

```
semgrep --version
```

Résultat attendu : Un numéro de version Semgrep.
Si la commande n'est pas reconnue et que je veux installer Semgrep avec Homebrew :

```
brew install semgrep
```

Pour vérifier le chemin de l'exécutable que mon terminal utilise :

```
which semgrep
```

Exemple de résultat :

```
/opt/homebrew/bin/semgrep
```

Cela me permet de savoir quel programme Semgrep est exécuté, notamment lorsque plusieurs environnements Python existent sur la machine.

2. Analyser uniquement le code Python de mon backend

Si je viens de modifier le backend FastAPI et que je veux rechercher des patterns potentiellement dangereux dans les fichiers Python de mon application :

```
semgrep scan --config auto app/
```

Résultat attendu : Semgrep affiche un résumé contenant notamment :

```
Scan Summary
Findings: ...
Rules run: ...
Targets scanned: ...
```

J'apprends à interpréter trois éléments :
Findings : nombre de constats signalés.
Rules run : nombre de règles exécutées.
Targets scanned : nombre de fichiers analysés.
Attention : un finding n'est pas automatiquement une vulnérabilité exploitable. Je dois examiner le code concerné.

3. Analyser mon backend et mon frontend ensemble

Si je veux rechercher des problèmes de sécurité dans toute la partie applicative de SecureShip, aussi bien Python que React/TypeScript :

```
semgrep scan --config auto app/ frontend/src/
```

Résultat attendu : Un rapport regroupant les constats détectés dans les fichiers analysés.
Dans notre première analyse SecureShip, nous avions observé :

```
Rules run: 452
Targets scanned: 25
Findings: 3
```

Les trois alertes concernaient la construction de requêtes SQL dans app/database.py.
L'analyse manuelle a montré que les expressions examinées utilisaient des identifiants et des fragments SQL contrôlés par le programme, tandis que les données utilisateur étaient passées par des paramètres SQLite.
Nous avons documenté ces constats comme probables faux positifs pour les expressions étudiées, et non comme une preuve générale

d'absence de SQL injection.

4. Empêcher qu'un scan Semgrep avec des constats non supprimés soit accepté par mon pipeline

Si je veux que Semgrep retourne un code d'erreur lorsqu'il détecte des constats non supprimés par une exception reconnue :

```
semgrep scan --config auto --error app/ frontend/src/
```

Résultat attendu :
Aucun constat bloquant : code de sortie 0.
Constats non supprimés : code de sortie non nul, généralement 1.
C'est exactement la commande utilisée dans le job semgrep de GitHub Actions.
Différence à mémoriser :

```
semgrep scan --config auto : analyse et affiche les constats.
semgrep scan --config auto --error : analyse et utilise également les constats pour faire échouer la commande.
```

5. Vérifier si Semgrep a réussi ou échoué

Si je viens d'exécuter Semgrep et que je veux connaître son code de sortie :

```
echo $?
```

Exemples de résultats :

```
0
```

La dernière commande s'est terminée avec succès.

```
1
```

La dernière commande s'est terminée avec une erreur ou un échec.
Important : echo $? doit être exécuté immédiatement après la commande que je veux vérifier, car une autre commande modifierait ce résultat.

6. Obtenir davantage de détails pendant l'analyse Semgrep

Si Semgrep ne scanne pas les fichiers que j'attendais ou si je veux obtenir davantage d'informations sur l'exécution :

```
semgrep scan --config auto --verbose app/ frontend/src/
```

Résultat attendu : Des informations supplémentaires sur les règles, les fichiers examinés et certains éléments du déroulement du scan.
Cette commande est utile pour diagnostiquer un résultat inattendu, mais elle n'est pas nécessaire à chaque analyse.

7. Exporter les résultats Semgrep dans un fichier JSON

Si je veux conserver les résultats d'un scan pour une analyse ultérieure ou un rapport de sécurité :

```
semgrep scan --config auto --json \
--output ~/Desktop/secureship-semgrep-report.json \
app/ frontend/src/
```

Résultat attendu : Un fichier secureship-semgrep-report.json apparaît sur mon Bureau.
Je peux y retrouver les constats, les règles correspondantes, les chemins de fichiers et les emplacements concernés.
Je conserve ce rapport en dehors du dépôt jusqu'à avoir vérifié qu'il ne contient aucune donnée qu'il ne faut pas publier.

8. Vérifier que ma correction Semgrep a réellement fonctionné

Si j'ai examiné une alerte, corrigé le code ou ajouté une exception précisément justifiée, et que je veux vérifier que le résultat a changé :

```
semgrep scan --config auto --error app/ frontend/src/
```

Résultat historique de SecureShip après notre triage :

```
Findings: 0 (0 blocking)
Rules run: 452
Targets scanned: 25
```

Cela signifie qu'aucun constat non supprimé n'a été signalé par cette analyse.
Ce que je dois retenir : nous n'avons pas désactivé Semgrep. Nous avons justifié des exceptions limitées aux expressions examinées dans app/database.py et conservé la protection du pipeline.

9. Vérifier que le mode bloquant fonctionne vraiment

Si je veux comprendre comment Semgrep réagit lorsqu'il trouve un problème sur un fichier volontairement vulnérable, je peux analyser un fichier d'exercice isolé, sans modifier l'application SecureShip :

```
semgrep scan --config auto --error --no-git-ignore \
/tmp/secureship-semgrep-demo.py
```

Condition : le fichier de démonstration doit exister et avoir été créé spécialement pour ce test.

Lors de notre démonstration locale, nous avions obtenu :

```
Findings: 2 (2 blocking)
Rules run: 290
Targets scanned: 1
```

Puis :

```
Code de sortie Semgrep : 1
```

Cela démontre qu'un constat peut entraîner l'échec du scan en mode bloquant.
Important : cet exercice local n'était pas une exécution GitHub Actions en échec et ne prouve pas que notre application contenait ces deux vulnérabilités.

Partie 2 — Gitleaks : Secret Detection

10. Vérifier si Gitleaks est disponible sur mon MacBook

Si je veux savoir si Gitleaks est installé et connaître sa version :

```
gitleaks version
```

Résultat attendu : La version de Gitleaks utilisée.
Si Gitleaks n'est pas installé, une méthode d'installation sur MacBook est :

```
brew install gitleaks
```

11. Vérifier si mon code actuel contient des secrets détectables

Si je veux analyser uniquement les fichiers actuels de mon backend pour rechercher des chaînes ressemblant à des mots de passe, des tokens ou des clés API :

```
gitleaks dir app/ --redact=100
```

Résultat attendu si aucun secret n'est détecté :

```
no leaks found
```

À comprendre :
dir : analyse les fichiers présents dans le répertoire ciblé.
app/ : limite l'analyse au backend.

```
--redact=100 : masque intégralement les secrets détectés dans les résultats pris en charge par l'outil.
```

Cette commande ne permet pas de savoir si un secret existait dans un ancien commit.

12. Rechercher des secrets dans l'historique Git de SecureShip

Si je veux analyser les commits enregistrés dans mon dépôt, et non seulement les fichiers actuels :

```
gitleaks git . --log-opts="--all" --redact=100
```

Résultat observé dans notre dépôt SecureShip :

```
no leaks found
```

À retenir :
git : analyse l'historique Git.
. : indique le dépôt courant.
–log-opts="–all" : inclut les références accessibles de l'historique Git.

```
--redact=100 : évite d'afficher les valeurs complètes détectées.
```

Le résultat signifie qu'aucun secret correspondant aux règles Gitleaks n'a été détecté dans le périmètre analysé.
Cela ne prouve pas que tous les secrets imaginables sont absents.

13. Comprendre pourquoi supprimer un secret d'un fichier ne suffit pas

Si un développeur a enregistré une clé API dans un commit Git, puis la supprime dans un commit suivant, je veux comprendre si l'ancienne clé peut toujours être retrouvée dans l'historique.
Notre démonstration a utilisé un dépôt de test distinct, avec un token fictif et non fonctionnel.
Premier résultat :

```
3 commits scanned
leaks found: 1
```

Après suppression du token dans la dernière version du fichier :

```
4 commits scanned
leaks found: 1
```

Ce que je comprends : le secret n'existe plus dans le fichier courant, mais il est encore présent dans l'historique Git.
C'est pourquoi nous analysons l'historique complet avec Gitleaks.
Si un véritable secret est exposé : il faut le révoquer et le remplacer immédiatement. Nettoyer un fichier ou l'historique ne rend pas automatiquement l'ancien secret inutilisable.

14. Conserver un rapport Gitleaks sans afficher de secrets en clair

Si je veux enregistrer les constats d'un scan d'historique dans un fichier JSON :

```
gitleaks git . --log-opts="--all" \
--redact=100 \
--report-format json \
--report-path ~/Desktop/secureship-gitleaks-report.json
```

Résultat attendu : Un rapport est créé sur le Bureau selon les résultats et le comportement de la version de Gitleaks utilisée.
Je dois contrôler son contenu avant de le partager.
À mémoriser : Gitleaks est notre contrôle de détection des secrets, et son job GitHub Actions parcourt l'historique Git accessible à l'exécution.

Partie 3 — Trivy : Container Vulnerability Scanning

15. Vérifier la version de Trivy

Si je veux vérifier que Trivy est installé sur mon MacBook :

```
trivy --version
```

Résultat attendu : Le numéro de version de Trivy.
Si je dois l'installer avec Homebrew :

```
brew install trivy
```

16. Construire une image SecureShip que je pourrai analyser avec Trivy

Si j'ai modifié mon Dockerfile ou les dépendances de l'application et que je veux analyser la nouvelle image plutôt qu'une ancienne :

```
docker build -t secureship:ci .
```

Résultat attendu : Docker construit une image appelée secureship :ci.
Cette image est celle que nous utilisons également dans notre workflow GitHub Actions.
Important : cette commande construit l'image ; elle ne supprime pas notre volume SQLite et ne modifie pas notre base locale.

17. Analyser toutes les vulnérabilités connues de mon image Docker

Si je veux savoir si les composants de mon image Docker possèdent des vulnérabilités connues, quelles sont leurs sévérités et quels paquets sont concernés :

```
trivy image --scanners vuln secureship:ci
```

Résultat attendu : Trivy examine l'image et affiche les vulnérabilités détectées, regroupées par composants et niveaux de sévérité.
Historique réel de SecureShip avant durcissement :

```
175 findings
```

Après réduction des dépendances runtime :

```
166 findings
```

Après la mise à jour Debian ciblée :

```
164 findings
```

Ces nombres correspondent à notre environnement et à la base de vulnérabilités de l'époque.

18. Afficher uniquement les vulnérabilités HIGH et CRITICAL

Si le rapport complet est long et que je veux me concentrer sur les vulnérabilités les plus sévères :

```
trivy image --scanners vuln \
--severity HIGH,CRITICAL \
secureship:ci
```

Résultat attendu : Seules les vulnérabilités classées HIGH ou CRITICAL sont affichées.
Dans notre dernier rapport local, la distribution complète était :

Sévérité	Nombre
CRITICAL	0
HIGH	44
MEDIUM	58
LOW	61
UNKNOWN	1
Total	164

À retenir : la sévérité aide à prioriser les investigations, mais elle ne suffit pas, seule, à déterminer le risque réel pour l'application.

19. Vérifier si mon image contient des vulnérabilités HIGH ou CRITICAL avec un correctif disponible

Si je veux que mon pipeline refuse une image contenant des vulnérabilités HIGH ou CRITICAL pour lesquelles Trivy connaît une version corrigée :

```
trivy image \
--scanners vuln \
--severity HIGH,CRITICAL \
--ignore-unfixed \
--exit-code 1 \
secureship:ci
```

Résultat attendu :
Code de sortie 0 : aucune vulnérabilité correspondant au filtre bloquant n'est détectée.
Code de sortie 1 : des vulnérabilités correspondant au filtre bloquant sont détectées.
C'est notre politique de blocage actuelle dans GitHub Actions.
Ce que je ne dois surtout pas confondre :
Une image peut contenir des dizaines de vulnérabilités dans le rapport complet et réussir ce gate si ces vulnérabilités n'ont pas de correctif disponible selon le scanner.
Dans SecureShip, les 44 HIGH du rapport complet n'ont pas été présentées comme corrigées simplement parce que le gate a réussi.

20. Exporter toutes les vulnérabilités de mon image dans un rapport JSON

Si je veux conserver une analyse complète pour identifier les paquets touchés et documenter les vulnérabilités :

```
trivy image \
--scanners vuln \
--format json \
--output ~/Desktop/secureship-trivy-report.json \
secureship:ci
```

Résultat attendu : Un rapport JSON est enregistré sur le Bureau.
Je peux l'utiliser pour analyser les identifiants de vulnérabilités, les paquets concernés et les informations de correction disponibles.
Dans GitHub Actions, nous utilisons également un rapport Trivy complet, publié comme artifact :

```
secureship-trivy-report
```

La rétention configurée est de 14 jours.

21. Savoir si le durcissement de mon image a réduit les constats

Si je viens de modifier mon Dockerfile, et que je veux savoir si le changement a réduit les vulnérabilités :
Je construis l'image corrigée avec docker build.
Je lance un nouveau scan complet Trivy.
Je compare les résultats à l'ancien rapport.
Exemple de notre travail :

Étape	Constats
Image initiale	175
Suppression des dépendances runtime inutiles	166
Mise à jour de liblzma5	164

Ce que je comprends : la réduction du nombre de constats est une mesure observable, mais elle ne prouve pas qu'une image est entièrement sécurisée.
Pour comparer rigoureusement deux images, il faut également tenir compte de leur architecture, des versions de scanner et de base de vulnérabilités, ainsi que de la date de l'analyse.

Partie 4 — SCA : Software Composition Analysis

La SCA n'est pas un programme unique. C'est une famille d'analyses qui permet d'examiner les composants logiciels et leurs dépendances pour identifier les vulnérabilités connues.
Dans SecureShip, nous utilisons principalement :

```
pip-audit pour Python.
npm audit pour le frontend React.
```

22. Installer pip-audit pour analyser mes dépendances Python

Si je veux installer l'outil qui permettra d'analyser les dépendances Python de mon projet :

```
python -m pip install pip-audit
```

Résultat attendu : L'outil pip-audit est installé dans l'environnement Python actif.
Pour vérifier son installation :

```
pip-audit --version
```

Je dois utiliser de préférence notre environnement virtuel .venv lors des vérifications locales.

23. Rechercher des vulnérabilités connues dans requirements.txt

Si je veux savoir si les dépendances Python déclarées dans requirements.txt possèdent des vulnérabilités connues :

```
pip-audit -r requirements.txt
```

Résultat de notre validation SecureShip :

```
No known vulnerabilities found
```

Cela signifie que l'outil n'a signalé aucune vulnérabilité connue pour le périmètre et les versions analysés à cette date.
Attention : cela ne signifie pas que les dépendances ne pourront jamais devenir vulnérables.
Cette commande est exécutée automatiquement dans notre job sca de GitHub Actions.

24. Vérifier les dépendances frontend, y compris celles utilisées pour développer et compiler

Si je veux effectuer un audit complet des dépendances npm de SecureShip, y compris les outils de développement comme Vite et TypeScript :
Depuis la racine du dépôt :

```
npm --prefix frontend audit
```

Résultat historique de notre analyse :

```
9 vulnerabilities
6 high
3 moderate
```

Ce résultat a montré que certaines dépendances utilisées pendant le développement ou la compilation nécessitaient une investigation.
Ce que je comprends : le fait qu'une dépendance soit réservée au développement ne signifie pas qu'elle est sans risque pour le poste de travail ou la chaîne de compilation.

25. Vérifier uniquement les dépendances npm de production

Si je veux lancer l'audit que nous avons configuré comme contrôle bloquant dans notre pipeline CI :

```
npm --prefix frontend audit --omit=dev
```

Résultat historique de SecureShip :

```
found 0 vulnerabilities
```

Différence essentielle :

```
npm audit : examine l'inventaire complet, y compris les dépendances de développement.
npm audit --omit=dev : exclut les dépendances de développement du périmètre de cet audit.
```

Notre job sca vérifie automatiquement la deuxième commande.
Les neuf constats de l'audit complet restent donc à suivre séparément.

26. Conserver un rapport JSON de l'audit npm complet

Si je veux garder les détails des vulnérabilités npm pour une future phase de correction :

```
npm --prefix frontend audit --json \
> ~/Desktop/secureship-npm-audit.json
```

Résultat attendu : Un rapport JSON apparaît sur mon Bureau.
La commande peut retourner un code d'erreur si des vulnérabilités sont signalées. Cela ne signifie pas automatiquement que le rapport n'a pas été généré.
Je peux l'utiliser pour identifier les paquets concernés et les chemins de dépendances.

27. Savoir quelles dépendances npm sont anciennes

Si je veux connaître les versions installées et les versions plus récentes disponibles dans le frontend :

```
npm --prefix frontend outdated
```

Résultat attendu : Un tableau indiquant notamment les versions Current, Wanted et Latest.
Cette commande m'aide à préparer un plan de mise à jour.
Précaution importante : nous n'avons pas exécuté aveuglément npm audit fix –force, car certaines corrections proposées pouvaient introduire des changements majeurs incompatibles avec notre projet.
Une correction SCA sérieuse doit être suivie d'un nouveau build, de tests et d'un nouvel audit.

Partie 5 — Checkov : Configuration Security

28. Vérifier que Checkov est installé

Si je veux savoir si Checkov est disponible sur mon MacBook :

```
checkov --version
```

Résultat attendu : Un numéro de version Checkov.
Pour l'installer dans un environnement Python :

```
python -m pip install checkov
```

Dans notre pipeline CI, la version utilisée est précisément :

```
checkov==3.3.20
```

29. Vérifier si mon Dockerfile respecte les bonnes pratiques analysées par Checkov

Si j'ai modifié le Dockerfile de SecureShip et que je veux savoir si Checkov y détecte des configurations risquées :

```
checkov -f Dockerfile --framework dockerfile
```

Résultat obtenu pendant notre validation :

```
Passed checks: 9
Failed checks: 0
```

Cela signifie que les contrôles Dockerfile appliqués par Checkov ont réussi.
À retenir : Checkov ne vérifie pas si tous les packages de l'image sont exempts de vulnérabilités. C'est notamment le rôle de Trivy.

30. Vérifier si mon pipeline GitHub Actions contient des configurations risquées

Si j'ai modifié .github/workflows/ci.yml et que je veux rechercher des problèmes de configuration dans les workflows GitHub Actions :

```
checkov -d .github/workflows --framework github_actions
```

Résultat historique de notre validation :

```
Passed checks: 159
Failed checks: 0
```

Ce résultat correspond au nombre d'instances de contrôles réussies. Cela ne signifie pas qu'il existe 159 garanties de sécurité indépendantes.
Checkov vérifie des règles portant sur les configurations prises en charge par l'outil.

31. Comprendre la différence entre Checkov et Trivy

Si je veux savoir quel outil utiliser selon ce que je cherche :

Si je veux…	Outil
Repérer une configuration Dockerfile risquée	`Checkov`
Repérer une configuration GitHub Actions risquée	`Checkov`
Rechercher les vulnérabilités connues des composants d'une image	`Trivy`
Rechercher des patterns de code potentiellement dangereux	`Semgrep`
Rechercher des secrets dans Git	`Gitleaks`
Rechercher des dépendances vulnérables Python ou npm	`pip-audit / npm audit`

Ces outils sont complémentaires ; ils ne vérifient pas les mêmes éléments.

Partie 6 — Actionlint et ShellCheck : Validation du workflow CI

32. Vérifier si Actionlint est installé

Si je veux savoir si Actionlint est disponible pour contrôler mes fichiers GitHub Actions :

```
actionlint -version
```

Résultat attendu : Le numéro de version.
Sur MacBook, je peux l'installer avec :

```
brew install actionlint
```

Pour installer également ShellCheck :

```
brew install shellcheck
```

33. Vérifier mon workflow GitHub Actions avant de faire un git push

Si j'ai modifié .github/workflows/ci.yml et que je veux rechercher des erreurs de syntaxe, de structure ou certaines erreurs dans les scripts shell intégrés :

```
actionlint .github/workflows/ci.yml
```

Résultat attendu si aucune anomalie n'est détectée : Aucun message d'erreur.
Dans SecureShip, Actionlint avait signalé une variable inutile dans une boucle shell :

```
SC2034
```

Nous avons remplacé l'ancienne variable i par _ dans la boucle de vérification de santé Docker.
Après correction, une nouvelle exécution d'Actionlint ne signalait plus d'anomalie.
Important : Actionlint et ShellCheck ont été utilisés localement. Ils ne constituent pas un huitième job de notre GitHub Actions actuel.

Partie 7 — Comprendre les résultats et les codes de sortie

34. Savoir si un outil de sécurité a réussi

Si je viens d'exécuter un scan et que je veux connaître son code de sortie :

```
echo $?
```

Interprétation générale :

Code	Signification courante
0	La commande a réussi selon ses règles et options
1	Échec ou constat entraînant un code non nul, selon l'outil
Autre valeur	Erreur ou situation particulière à examiner dans la documentation de l'outil

Attention : je dois toujours consulter le message du scanner. Un code 0 ne signifie pas nécessairement « aucune vulnérabilité ».
Par exemple :
Un scan Semgrep sans –error peut réussir même s'il affiche des constats.
Le gate Trivy avec –ignore-unfixed peut réussir alors que le rapport complet contient des vulnérabilités non corrigées.
Un audit npm complet peut retourner un code non nul parce qu'il a identifié des vulnérabilités.
Lorsque je veux modifier le code ou les dépendances de SecureShip, je dois comprendre cette succession de contrôles.

35. Vérifier que les tests de mon backend réussissent

Si je viens de modifier le code Python et que je veux vérifier les comportements de mon application et les tests de sécurité :

```
python -m pytest -q
```

Résultat historique :

```
53 passed
```

36. Vérifier que le frontend fonctionne après mes modifications

Si j'ai modifié React ou les dépendances JavaScript et que je veux vérifier que le build fonctionne :

```
npm --prefix frontend run build
```

Résultat attendu : TypeScript et Vite terminent la compilation sans erreur.

37. Vérifier que mon code ne déclenche pas les règles Semgrep bloquantes

Si mes tests passent et que je veux analyser mon code Python et React :

```
semgrep scan --config auto --error app/ frontend/src/
```

Résultat attendu : Aucun constat bloquant pour réussir ce contrôle.

38. Vérifier les dépendances Python et npm

Si je veux rechercher des vulnérabilités connues dans les dépendances :

```
pip-audit -r requirements.txt
```

Puis :

```
npm --prefix frontend audit --omit=dev
```

Résultat attendu : Les audits se terminent avec succès selon les périmètres configurés.
Je conserve également un suivi du résultat de l'audit npm complet.

39. Vérifier que mon dépôt ne contient pas de secrets détectables dans son historique

Si je veux analyser mon historique Git avant de publier :

```
gitleaks git . --log-opts="--all" --redact=100
```

Résultat attendu dans notre état validé :

```
no leaks found
```

40. Vérifier mes configurations Docker et GitHub Actions

Si je veux vérifier que mes modifications de configuration respectent les contrôles Checkov :

```
checkov -f Dockerfile --framework dockerfile
```

Puis :

```
checkov -d .github/workflows --framework github_actions
```

Si je veux également détecter certaines erreurs dans mon workflow GitHub Actions :

```
actionlint .github/workflows/ci.yml
```

41. Construire puis analyser mon image Docker

Si je veux vérifier la sécurité de l'image réellement construite à partir du nouveau code :

```
docker build -t secureship:ci .
```

Puis :

```
trivy image --scanners vuln secureship:ci
```

Pour appliquer notre politique de blocage des vulnérabilités HIGH et CRITICAL ayant un correctif disponible :

```
trivy image --scanners vuln \
--severity HIGH,CRITICAL \
--ignore-unfixed --exit-code 1 \
secureship:ci
```

42. Publier mon travail après les vérifications

Si tous les changements sont prêts, correctement examinés et acceptables pour le projet, je peux préparer mon commit Git.
Après publication sur GitHub, notre pipeline exécute automatiquement les contrôles définis dans .github/workflows/ci.yml.
Dans SecureShip, les sept jobs sont :
test — Pytest.
frontend — TypeScript et Vite.

```
docker — Build, smoke tests et Trivy.
semgrep — SAST.
gitleaks — Secret scanning.
```

sca — pip-audit et npm audit de production.

```
checkov — Configuration security.
```

Le job Docker attend la réussite de test et frontend. Les autres jobs de sécurité peuvent s'exécuter indépendamment.
Notre dernière validation GitHub Actions : 7 jobs réussis sur le commit documentaire d3fa9c0.

Résumé — Les commandes de sécurité à mémoriser

Si je veux…	Commande
Connaître la version de Semgrep	`semgrep --version`
Analyser mon code avec Semgrep	`semgrep scan --config auto app/ frontend/src/`
Bloquer si Semgrep détecte un constat non supprimé	`semgrep scan --config auto --error app/ frontend/src/`
Obtenir plus de détails Semgrep	`semgrep scan --config auto --verbose app/ frontend/src/`
Analyser les fichiers actuels du backend avec Gitleaks	`gitleaks dir app/ --redact=100`
Rechercher des secrets dans l'historique Git	`gitleaks git . --log-opts="--all" --redact=100`
Connaître la version de Trivy	`trivy --version`
Construire l'image à analyser	`docker build -t secureship:ci .`
Analyser mon image Docker	`trivy image --scanners vuln secureship:ci`
Voir seulement les HIGH et CRITICAL	`trivy image --scanners vuln --severity HIGH,CRITICAL secureship:ci`
Appliquer notre gate Trivy	`trivy image --scanners vuln --severity HIGH,CRITICAL --ignore-unfixed --exit-code 1 secureship:ci`
Auditer les dépendances Python	`pip-audit -r requirements.txt`
Auditer toutes les dépendances npm	`npm --prefix frontend audit`
Auditer les dépendances npm de production	`npm --prefix frontend audit --omit=dev`
Vérifier mon Dockerfile avec Checkov	`checkov -f Dockerfile --framework dockerfile`
Vérifier GitHub Actions avec Checkov	`checkov -d .github/workflows --framework github_actions`
Vérifier la syntaxe et certains scripts du workflow	`actionlint .github/workflows/ci.yml`
Connaître le code de sortie de la dernière commande	`echo $?`

Les six notions essentielles à retenir

SAST — Static Application Security Testing
Je veux analyser le code de mon application sans avoir besoin de l'exécuter. Dans SecureShip, j'utilise Semgrep.
Secret Scanning
Je veux vérifier si des clés, tokens ou mots de passe ont été enregistrés dans le code ou dans l'historique Git. J'utilise Gitleaks.
SCA — Software Composition Analysis
Je veux connaître les vulnérabilités connues des bibliothèques et dépendances utilisées par mon projet. J'utilise pip-audit et npm audit.
Container Vulnerability Scanning
Je veux analyser les composants contenus dans mon image Docker, y compris certains paquets système. J'utilise Trivy.
Configuration Security
Je veux détecter des configurations risquées dans mon Dockerfile ou mes workflows GitHub Actions. J'utilise Checkov.
CI — Continuous Integration
Je veux que les tests et analyses s'exécutent automatiquement lorsque le code est publié ou proposé dans une Pull Request. J'utilise GitHub Actions.

Conclusion

Grâce à SecureShip, je comprends maintenant que la sécurité d'une application ne dépend pas d'un seul scanner.
Je dois examiner le code, les dépendances, les secrets, l'image Docker et les configurations. Puis je dois interpréter les résultats, corriger les problèmes réels, justifier les exceptions et conserver des preuves de validation.
La démarche DevSecOps à mémoriser :
Build → Test → Scan → Investigate → Remediate → Re-scan → Validate → Commit/Push → CI.
Si un contrôle échoue, je dois examiner la cause, appliquer la correction appropriée et refaire les vérifications.
Un pipeline vert signifie que les contrôles configurés ont réussi ; il ne signifie jamais que toutes les vulnérabilités ont disparu.